

Documentation Technique Complète — API Webservice PrestaShop

8.2.6

Table des matières

- 1. C'est quoi l'API Webservice PrestaShop ?
 - 2. Comment ça marche concrètement ? (Architecture)
 - 3. Le Dispatcher : le point d'entrée unique
 - 4. L'Authentification : comment se connecter à l'API
 - 5. Les formats de réponse (XML et JSON)
 - 6. Les méthodes HTTP (CRUD)
 - 7. Liste complète de toutes les ressources API
 - 8. PRODUITS — Tout savoir sur l'API Products
 - 9. CLIENTS — Tout savoir sur l'API Customers
 - 10. COMMANDES — Tout savoir sur l'API Orders
 - 11. PANIERS — Tout savoir sur l'API Carts
 - 12. Le moteur interne : les classes Webservice
 - 13. Arborescence complète des fichiers
 - 14. Exemples concrets de requêtes (copier-coller)
 - 15. Scénarios complets pour la NewApp
-

1. C'est quoi l'API Webservice PrestaShop ?

Explication simple

Imagine que PrestaShop est un **magasin physique**. L'API Webservice c'est comme une **porte de service** qui permet à un programme externe (ta future NewApp) de :

- Lire les informations du magasin (produits, clients, commandes...)
- Ajouter de nouvelles données (créer un client, une commande...)
- Modifier des données existantes (changer un prix, un statut...)
- Supprimer des données (retirer un produit du catalogue...)

Sans cette API, ta NewApp ne peut PAS communiquer avec PrestaShop. Elle serait comme une application isolée, incapable d'accéder aux données de la boutique.

Caractéristiques techniques

Caractéristique	Valeur
Type d'API	REST (Representational State Transfer)
Format principal	XML (par défaut)
Format alternatif	JSON
Authentification	Clé API (32 caractères)
Protocole	HTTP/HTTPS
URL de base	http://localhost:8080/api/
Version PrestaShop	8.2.6
Version PHP	8.1

Différence entre API REST et une page web classique

PAGE WEB CLASSIQUE (ce que tu vois dans ton navigateur) :

Navigateur → `http://localhost:8080/fr/` → Reçoit du HTML/CSS
(visuellement : la page d'accueil avec les produits en image)

API WEBSERVICE (ce que ta NewApp utilisera) :

Programme → `http://localhost:8080/api/products` → Reçoit du XML/JSON
(des données brutes : id, nom, prix, description...)

La page web c'est pour les humains. L'API c'est pour les programmes.

2. Comment ça marche concrètement ? (Architecture)

Schéma du flux complet d'une requête API



Ta NewApp
(Frontend)

Reçoit les données et les affiche

Concepts clés à comprendre

ObjectModel : C'est la classe de base de TOUTES les entités PrestaShop (Product, Customer, Order, Cart...). Chaque ObjectModel :

- Correspond à une table dans la base de données MySQL
- Sait se sauvegarder (`add()`), se modifier (`update()`), se supprimer (`delete()`)
- Définit ses propres `$webserviceParameters` pour décrire quels champs exposer via l'API
- Définit ses propres `$definition` avec les types, validations et contraintes de chaque champ

Pattern Singleton : `WebserviceRequest` n'existe qu'en UN seul exemplaire pendant toute la requête. On y accède via `WebserviceRequest::getInstance()` . Ça veut dire que peu importe combien de fois on appelle cette méthode, on obtient toujours le même objet.

xlink_resource : Quand un champ fait référence à une autre entité (ex: `id_customer` dans une commande fait référence à un client), PrestaShop ajoute un lien hypertexte XML vers cette ressource. C'est comme un lien cliquable dans le XML :

```
<id_customer xlink:href="http://localhost:8080/api/customers/3">3</id_customer>
```

3. Le Dispatcher : le point d'entrée unique

Fichier : `prestashop/webservice/dispatcher.php`

C'est le PREMIER fichier PHP exécuté quand une requête arrive sur `/api/` . C'est le "gardien de la porte".

Ce qu'il fait étape par étape (code simplifié) :

```
// ÉTAPE 1 : Charge toute la configuration de PrestaShop
// (connexion BDD, constantes, autoloader des classes...)
require_once dirname(__FILE__) . '/../config/config.inc.php';

// ÉTAPE 2 : Initialise le contexte
// - Un panier vide (certaines fonctions en ont besoin)
// - Le container Symfony (injection de dépendances)
// - La devise par défaut
Context::getContext()->cart = new Cart();
Context::getContext()->container = ContainerBuilder::getContainer('webservice', _PS_MODE_DEV_);
Context::getContext()->currency = Currency::getDefaultCurrency();

// ÉTAPE 3 : Récupère la clé d'authentification
// 2 façons de la passer :
if (isset($_SERVER['PHP_AUTH_USER'])) {
    // Méthode 1 : HTTP Basic Auth (header "Authorization: Basic ...")
    $key = $_SERVER['PHP_AUTH_USER'];
} elseif (isset($_GET['ws_key'])) {
    // Méthode 2 : Paramètre GET dans l'URL (?ws_key=...)
    $key = $_GET['ws_key'];
} else {
    // Pas de clé → erreur 401 Unauthorized
    die('401 Unauthorized');
}

// ÉTAPE 4 : Si la requête est POST/PUT/PATCH, lit le XML envoyé dans le body
if (in_array($_SERVER['REQUEST_METHOD'], ['POST', 'PUT', 'PATCH'])) {
    $input_xml = file_get_contents('php://input');
}

// ÉTAPE 5 : Crée l'instance de WebserviceRequest et exécute la requête
$request = WebserviceRequest::getInstance();
$result = $request->fetch($key, $method, $url, $params, $bad_class_name, $input_xml);
// $result contient : ['headers' => [...], 'content' => '<xml>...</xml>', 'type' => 'xml']

// ÉTAPE 6 : Envoie les headers HTTP de réponse
foreach ($result['headers'] as $header) {
    header($header); // Ex: "HTTP/1.1 200 OK", "Content-Type: text/xml"
}

// ÉTAPE 7 : Affiche le contenu (XML ou JSON)
echo $result['content'];
```

URL de base de l'API :

```
http://localhost:8080/api/
```

Comment Apache sait rediriger vers ce fichier ? Le fichier `.htaccess` à la racine de PrestaShop contient une règle de réécriture (mod_rewrite) qui dit : "Toute URL commençant par `/api/` doit être redirigée vers `webservice/dispatcher.php` "

4. L'Authentification : comment se connecter à l'API

Comment ça fonctionne

Pour utiliser l'API, tu as besoin d'une **clé API**. C'est une chaîne de 32 caractères hexadécimaux. Exemple : `X7R2K9M4N8P5Q3S6T1W0Y4Z7A2B5C8D1`

Cette clé est stockée dans la base de données MySQL, dans la table `ps_webservice_account` .

Fichier : `prestashop/classes/webservice/WebserviceKey.php`

C'est la classe PHP qui gère les clés API :

```

class WebserviceKeyCore extends ObjectModel
{
    public $key;           // La clé elle-même (32 chars)
    public $active = true; // Si la clé est active ou désactivée
    public $description;    // Description libre (ex: "Clé pour la NewApp")

    public static $definition = [
        'table' => 'webservice_account',    // Table MySQL : ps_webservice_account
        'primary' => 'id_webservice_account', // Clé primaire
        'fields' => [
            'active' => ['type' => self::TYPE_BOOL, 'validate' => 'isBool'],
            'key' => ['type' => self::TYPE_STRING, 'required' => true, 'size' => 32],
            'description' => ['type' => self::TYPE_STRING, 'size' => 4194303],
        ],
    ];
}

```

Les 2 tables en base de données :

Table `ps_webservice_account` (stocke les clés) :

Colonne	Type	Description
<code>id_webservice_account</code>	int	ID unique de la clé
<code>key</code>	varchar(32)	La clé API elle-même
<code>active</code>	tinyint	1 = active, 0 = désactivée
<code>description</code>	text	Description libre

Table `ps_webservice_permission` (stocke les droits de chaque clé) :

Colonne	Type	Description
<code>id_webservice_permission</code>	int	ID unique
<code>id_webservice_account</code>	int	Référence vers la clé
<code>resource</code>	varchar	Nom de la ressource (ex: "products")
<code>method</code>	varchar	Méthode autorisée (GET, POST, PUT, DELETE)

Exemple concret : Si tu veux que ta clé puisse LIRE les produits mais pas les SUPPRIMER :

- Il y aura une ligne `resource=products, method=GET` → AUTORISÉ
- Il n'y aura PAS de ligne `resource=products, method=DELETE` → INTERDIT

Comment passer la clé dans une requête

Méthode 1 : Paramètre GET (la plus simple pour tester dans le navigateur) :

```
http://localhost:8080/api/products?ws_key=X7R2K9M4N8P5Q3S6T1W0Y4Z7A2B5C8D1
```

Méthode 2 : HTTP Basic Auth (recommandée en production, plus sécurisée) :

```

// La clé est mise comme "username", le mot de passe est vide
Authorization: Basic WDdSMks5TTR0OFAgNVEzUzZUMVcwWTRaN0EyQjVD0EQxOg==
// (c'est base64("X7R2K9M4N8P5Q3S6T1W0Y4Z7A2B5C8D1:"))

```

Méthode 3 : En JavaScript (pour ta NewApp) :

```
// Avec fetch
const API_KEY = 'X7R2K9M4N8P5Q3S6T1W0Y4Z7A2B5C8D1';
const response = await fetch('http://localhost:8080/api/products?output_format=JSON', {
  headers: {
    'Authorization': 'Basic ' + btoa(API_KEY + ':')
  }
});
const data = await response.json();
```

Comment créer une clé API dans le Back Office

1. Connecte-toi au Back Office : `http://localhost:8080/admin/`
2. Va dans le menu **Paramètres avancés** > **Webservice**
3. Active le Webservice (bouton ON en haut)
4. Clique "Ajouter une nouvelle clé webservice"
5. Remplis :
 - **Clé** : Cliquer "Générer" (il génère automatiquement 32 caractères)
 - **Description** : "Clé pour la NewApp"
 - **Statut** : Activé
 - **Permissions** : Cocher les ressources et méthodes que tu autorises
6. Sauvegarde

5. Les formats de réponse (XML et JSON)

XML (format par défaut)

Quand tu fais `GET /api/products/1`, tu reçois par défaut du XML :

```
<?xml version="1.0" encoding="UTF-8"?>
<prestashop xmlns:xlink="http://www.w3.org/1999/xlink">
  <product>
    <id>1</id>
    <id_manufacturer xlink:href="http://localhost:8080/api/manufacturers/1">1</id_manufacturer>
    <id_category_default xlink:href="http://localhost:8080/api/categories/4">4</id_category_default>
    <active>1</active>
    <price>23.900000</price>
    <name>
      <language id="1" xlink:href="http://localhost:8080/api/languages/1">
        T-shirt délavé
      </language>
    </name>
    <description>
      <language id="1">Description du produit...</language>
    </description>
    <associations>
      <categories>
        <category><id>4</id></category>
        <category><id>5</id></category>
      </categories>
    </associations>
  </product>
</prestashop>
```

Points importants sur le XML :

- La racine est toujours `<prestashop>`
- Les liens vers d'autres ressources utilisent `xlink:href` (liens cliquables vers les APIs liées)
- Les champs multilingues ont un sous-élément `<language id="X">` pour chaque langue
- Les associations (relations) sont dans un bloc `<associations>`

JSON (format alternatif)

Pour recevoir du JSON, ajoute `?output_format=JSON` :

```
{
  "product": {
    "id": 1,
    "id_manufacturer": "1",
    "id_category_default": "4",
    "active": "1",
    "price": "23.900000",
    "name": [
      { "id": "1", "value": "T-shirt délavé" }
    ],
    "associations": {
      "categories": [
        { "id": "4" },
        { "id": "5" }
      ]
    }
  }
}
```

Comment choisir le format :

Méthode	Exemple
Paramètre GET	<code>?output_format=JSON</code>
Header HTTP	<code>Io-Format: JSON</code>
Header HTTP	<code>Output-Format: JSON</code>
Par défaut (rien)	XML

Fichiers qui gèrent le formatage :

Fichier	Rôle
<code>classes/webservice/WebserviceOutputBuilder.php</code>	Organise les données (vue liste ou détail, profondeur, champs à afficher)
<code>classes/webservice/WebserviceOutputXML.php</code>	Convertit les données PHP en XML valide avec namespaces xlink
<code>classes/webservice/WebserviceOutputJSON.php</code>	Convertit les données PHP en JSON
<code>classes/webservice/WebserviceOutputInterface.php</code>	Interface commune que XML et JSON doivent respecter

6. Les méthodes HTTP (CRUD)

Qu'est-ce que CRUD ?

CRUD = Create, Read, Update, Delete. Ce sont les 4 opérations de base sur une donnée.

Opération	Méthode HTTP	Ce que ça fait concrètement	Code retour	Exemple
Lire	<code>GET</code>	Récupère les données sans rien modifier	200 OK	<code>GET /api/products</code>
Vérifier	<code>HEAD</code>	Comme GET mais renvoie SEULEMENT les headers (pas de body)	200 OK	<code>HEAD /api/products/1</code>
Créer	<code>POST</code>	Crée une NOUVELLE ressource dans la BDD	201 Created	<code>POST /api/products</code> + XML
Remplacer	<code>PUT</code>	Remplace TOUS les champs d'une ressource	200 OK	<code>PUT /api/products/1</code> + XML complet
Modifier	<code>PATCH</code>	Modifie SEULEMENT les champs envoyés	200 OK	<code>PATCH /api/products/1</code> + XML partiel

Opération	Méthode HTTP	Ce que ça fait concrètement	Code retour	Exemple
Supprimer	DELETE	Supprime la ressource de la BDD	200 OK	DELETE /api/products/1

Différence entre PUT et PATCH (IMPORTANT !)

Produit actuel en BDD : { id: 1, name: "T-shirt", price: 29.99, active: 1 }

PUT avec { id: 1, price: 19.99 }

→ Résultat : { id: 1, name: "", price: 19.99, active: 0 }

⚠ DANGER ! Les champs non envoyés (name, active) sont RÉINITIALISÉS

PATCH avec { id: 1, price: 19.99 }

→ Résultat : { id: 1, name: "T-shirt", price: 19.99, active: 1 }

✅ SAFE ! Seul price est modifié, le reste ne bouge pas

En résumé : utilise PATCH pour les modifications simples, PUT seulement si tu envoies TOUS les champs.

Ressources avec des restrictions de méthodes

Certaines ressources interdisent certaines méthodes :

Ressource	Méthodes interdites	Pourquoi
search	PUT, POST, PATCH, DELETE	La recherche est en lecture seule, tu ne "crées" pas un résultat de recherche
stock_availability	POST, DELETE	On ne crée pas un stock, on modifie la quantité existante
stock_movements	PUT, POST, PATCH, DELETE	C'est un historique, on ne le modifie jamais
warehouses	DELETE	Protection : on ne supprime pas un entrepôt via l'API

7. Liste complète de toutes les ressources API

Tout est défini dans prestashop/classes/webservice/WebserviceRequest.php , méthode getResources() (ligne 283).

Catalogue (Produits)

Ressource	URL	Explication	Classe	Fichier
products	/api/products	Les produits (nom, prix, stock, images...)	Product	classes/Product.php
categories	/api/categories	L'arbre des catégories (Vêtements > T-shirts)	Category	classes/Category.php
combinations	/api/combinations	Les déclinaisons (Taille S Rouge, Taille M Bleu...)	Combination	classes/Combination.php
product_features	/api/product_features	Les caractéristiques (ex: "Matière")	Feature	classes/Feature.php
product_feature_values	/api/product_feature_values	Leurs valeurs (ex: "Coton")	FeatureValue	classes/FeatureValue.php
product_options	/api/product_options	Les groupes d'attributs (ex: "Taille")	AttributeGroup	classes/AttributeGroup.php
product_option_values	/api/product_option_values	Leurs valeurs (ex: "S", "M", "L")	ProductAttribute	classes/ProductAttribute.php
manufacturers	/api/manufacturers	Les marques	Manufacturer	classes/Manufacturer.php

Ressource	URL	Explication	Classe	Fichier
suppliers	/api/suppliers	Les fournisseurs	Supplier	classes/Supplier.php
tags	/api/tags	Les tags des produits	Tag	classes/Tag.php
specific_prices	/api/specific_prices	Les promotions et prix spéciaux	SpecificPrice	classes/SpecificPrice.php

Clients

Ressource	URL	Explication	Classe	Fichier
customers	/api/customers	Les comptes clients	Customer	classes/Customer.php
addresses	/api/addresses	Les adresses de livraison/facturation	Address	classes/Address.php
groups	/api/groups	Les groupes clients (Visiteur, Client, VIP)	Group	classes/Group.php
guests	/api/guests	Les visiteurs non connectés	Guest	classes/Guest.php

Commandes et Paniers

Ressource	URL	Explication	Classe	Fichier
carts	/api/carts	Les paniers (avant paiement)	Cart	classes/Cart.php
cart_rules	/api/cart_rules	Les codes promo et réductions	CartRule	classes/CartRule.php
orders	/api/orders	Les commandes validées	Order	classes/order/Order.php
order_details	/api/order_details	Les lignes de commande	OrderDetail	classes/order/OrderDetail.php
order_histories	/api/order_histories	L'historique des statuts	OrderHistory	classes/order/OrderHistory.php
order_carriers	/api/order_carriers	Le transporteur de la commande	OrderCarrier	classes/order/OrderCarrier.php
order_invoices	/api/order_invoices	Les factures	OrderInvoice	classes/order/OrderInvoice.php
order_payments	/api/order_payments	Les paiements	OrderPayment	classes/order/OrderPayment.php
order_states	/api/order_states	Les statuts possibles	OrderState	classes/order/OrderState.php
order_slip	/api/order_slip	Les avoirs/remboursements	OrderSlip	classes/order/OrderSlip.php

Livraison, Taxes, Stock, Configuration

Ressource	URL	Explication	Classe	Fichier
carriers	/api/carriers	Transporteurs (Colissimo, UPS...)	Carrier	classes/Carrier.php
taxes	/api/taxes	Taux de TVA (20%, 10%...)	Tax	classes/tax/Tax.php
currencies	/api/currencies	Devises (EUR, USD...)	Currency	classes/Currency.php
stock_availables	/api/stock_availables	Quantités en stock	StockAvailable	classes/stock/StockAvailable.php
languages	/api/languages	Langues de la boutique	Language	classes/Language.php
countries	/api/countries	Liste des pays	Country	classes/Country.php

Ressource	URL	Explication	Classe	Fichier
<code>configurations</code>	<code>/api/configurations</code>	Paramètres de la boutique	<code>Configuration</code>	<code>classes/Configuration.php</code>
<code>images</code>	<code>/api/images</code>	Upload/gestion des images	Spécifique	<code>classes/webservice/WebserviceSpecificManagementImages.php</code>
<code>search</code>	<code>/api/search?query=xxx</code>	Recherche (GET uniquement)	Spécifique	<code>classes/webservice/WebserviceSpecificManagementSearch.php</code>

8. PRODUITS — Tout savoir sur l'API Products

Vue d'ensemble

Info	Valeur
URL	<code>/api/products</code>
Classe	<code>ProductCore</code> → fichier <code>classes/Product.php</code> (8491 lignes)
Table BDD	<code>ps_product</code> + <code>ps_product_shop</code> + <code>ps_product_lang</code>
Clé primaire	<code>id_product</code>

Tous les champs

Champs simples :

Champ	Type	Description
<code>id</code>	int	Identifiant unique
<code>id_manufacturer</code>	int	Lien vers la marque → <code>/api/manufacturers/{id}</code>
<code>id_supplier</code>	int	Lien vers le fournisseur → <code>/api/suppliers/{id}</code>
<code>id_category_default</code>	int	Catégorie principale → <code>/api/categories/{id}</code>
<code>id_default_image</code>	int	Image principale (getter: <code>getCoverWs</code> , setter: <code>setCoverWs</code>)
<code>id_default_combination</code>	int	Déclinaison par défaut → <code>/api/combinations/{id}</code>
<code>id_tax_rules_group</code>	int	Groupe de taxes → <code>/api/tax_rule_groups/{id}</code>
<code>type</code>	string	<code>simple</code> (normal), <code>pack</code> (lot), <code>virtual</code> (dématérialisé)
<code>active</code>	0/1	Visible sur la boutique ?
<code>price</code>	float	Prix HT (ex: 23.90)
<code>wholesale_price</code>	float	Prix d'achat (marge = price - wholesale_price)
<code>reference</code>	string	Référence interne du produit
<code>ean13</code>	string	Code-barres EAN13
<code>weight</code>	float	Poids en kg
<code>quantity</code>	int	Stock (LECTURE SEULE via l'API, modifier via <code>stock_avalables</code>)
<code>minimal_quantity</code>	int	Quantité minimum de commande
<code>date_add</code>	datetime	Date de création
<code>date_upd</code>	datetime	Dernière modification

Champ	Type	Description
<code>position_in_category</code>	int	Position d'affichage dans la catégorie
<code>manufacturer_name</code>	string	Nom de la marque (lecture seule)

Champs traduits (une valeur par langue) :

Champ	Type	Description
<code>name</code>	string	Nom du produit ("T-shirt délavé")
<code>description</code>	HTML	Description longue
<code>description_short</code>	HTML	Description courte
<code>meta_title</code>	string	Titre SEO
<code>meta_description</code>	string	Description SEO
<code>link_rewrite</code>	string	URL (ex: "t-shirt-delave")
<code>available_now</code>	string	Message si en stock
<code>available_later</code>	string	Message si hors stock

Associations (données liées au produit) :

Association	Ce qu'elle contient	Exemple
<code>categories</code>	IDs des catégories du produit	Le T-shirt est dans "Vêtements" (4) et "T-shirts" (5)
<code>images</code>	IDs des images uploadées	3 photos du T-shirt
<code>combinations</code>	IDs des déclinaisons	Taille S, M, L / Couleur Rouge, Bleu
<code>product_features</code>	Caractéristiques	Matière: Coton, Composition: 100% coton
<code>tags</code>	Tags pour la recherche	"t-shirt", "été", "casual"
<code>stock_abilities</code>	Stock par déclinaison	S=10, M=25, L=5
<code>accessories</code>	Produits complémentaires	"Vous aimerez aussi : ce jean"
<code>product_bundle</code>	Composition d'un pack	Pack = 2x T-shirts + 1x Casquette

Configuration webservice dans le code :

```
// classes/Product.php (ligne 575)
protected $webserviceParameters = [
    'objectMethods' => [
        'add' => 'addWs',      // POST appelle addWs() au lieu de add()
        'update' => 'updateWs', // PUT/PATCH appelle updateWs() au lieu de update()
    ],
    'objectNodeNames' => 'products', // Nom dans le XML : <products>
    'fields' => [
        'id_manufacturer' => ['xlink_resource' => 'manufacturers'],
        // ...chaque champ avec sa config
    ],
    'associations' => [
        'categories' => ['resource' => 'category', 'fields' => ['id' => ['required' => true]]],
        'images' => ['resource' => 'image', 'fields' => ['id' => []]],
        // ...chaque association
    ],
];
```

Fichiers liés aux produits :

classes/Product.php	← Modèle principal (définition, webserviceParameters, logique)
controllers/admin/AdminProductsController.php	← Controller Back Office (ancien système)
src/Core/Domain/Product/	← Nouveau système DDD (Commands, Queries, ValueObjects)
src/Adapter/Product/	← Pont entre ancien et nouveau système

9. CLIENTS — Tout savoir sur l'API Customers

Vue d'ensemble

Info	Valeur
URL	/api/customers
Classe	CustomerCore → fichier classes/Customer.php (1566 lignes)
Table BDD	ps_customer
Clé primaire	id_customer

Tous les champs

Champ	Type	Requis	Lecture seule	Description
id	int	Auto	Oui	Identifiant unique
id_default_group	int	Non	Non	Groupe par défaut → /api/groups
id_lang	int	Non	Non	Langue du client → /api/languages
id_gender	int	Non	Non	1 = Monsieur, 2 = Madame
firstname	string	Oui	Non	Prénom
lastname	string	Oui	Non	Nom
email	string	Oui	Non	Email (doit être unique !)
passwd	string	Oui	Non	Mot de passe (hashé automatiquement par setWsPasswd)
birthday	date	Non	Non	Date de naissance (YYYY-MM-DD)
newsletter	0/1	Non	Non	Inscrit à la newsletter ?
optin	0/1	Non	Non	Accepte les offres partenaires ?
company	string	Non	Non	Société (B2B)
siret	string	Non	Non	SIRET (B2B France)
website	string	Non	Non	Site web
active	0/1	Non	Non	Compte activé ?
is_guest	0/1	Non	Non	Compte invité (sans inscription) ?
note	string	Non	Non	Note interne (visible qu'en Back Office)
secure_key	string	Non	Oui	Clé de sécurité (generée automatiquement, impossible à modifier via API)
last_passwd_gen	datetime	Non	Oui	Dernière génération de mot de passe (lecture seule)
date_add	datetime	Auto	Non	Date de création du compte
date_upd	datetime	Auto	Non	Dernière modification

Association : `groups` → liste des groupes du client (Visiteur, Client, VIP...)

Points importants :

- 1. **Mot de passe** : Tu envoies le mot de passe en clair dans le XML, PrestaShop le hash automatiquement via la méthode `setWsPasswd()` . Tu ne peux JAMAIS lire le mot de passe en clair.
- 2. **Email unique** : Si tu essaies de créer un client avec un email déjà utilisé → erreur.
- 3. **Soft delete** : Supprimer un client met juste `deleted=1` en BDD, la ligne reste.

Fichiers liés :

<code>classes/</code>	← Modèle principal
<code>src/Core/Domain/</code>	← DDD (Commands, Queries, ValueObjects)
<code>src/Adapter/</code>	← Adaptateurs
<code>src/Core/</code>	← Services métier

10. COMMANDES — Tout savoir sur l'API Orders

Vue d'ensemble

Info	Valeur
URL	<code>/api/orders</code>
Classe	<code>OrderCore</code> → fichier <code>classes/order/Order.php</code> (2877 lignes)
Table BDD	<code>ps_orders</code>
Clé primaire	<code>id_order</code>

Tous les champs

Champ	Type	Requis	Description
<code>id</code>	int	Auto	Identifiant unique
<code>id_address_delivery</code>	int	Oui	Adresse de livraison → <code>/api/addresses</code>
<code>id_address_invoice</code>	int	Oui	Adresse de facturation → <code>/api/addresses</code>
<code>id_cart</code>	int	Oui	Panier source → <code>/api/carts</code>
<code>id_currency</code>	int	Oui	Devise → <code>/api/currencies</code>
<code>id_lang</code>	int	Oui	Langue → <code>/api/languages</code>
<code>id_customer</code>	int	Oui	Client → <code>/api/customers</code>
<code>id_carrier</code>	int	Oui	Transporteur → <code>/api/carriers</code>
<code>current_state</code>	int	Non	Statut actuel (setter spécial: <code>setWsCurrentState</code>) → <code>/api/order_states</code>
<code>module</code>	string	Oui	Module de paiement (ex: "ps_wirepayment")
<code>payment</code>	string	Oui	Nom du paiement affiché (ex: "Virement bancaire")
<code>total_paid</code>	float	Oui	Montant total à payer
<code>total_paid_tax_incl</code>	float	Non	Total TTC
<code>total_paid_tax_excl</code>	float	Non	Total HT
<code>total_paid_real</code>	float	Oui	Montant réellement encaissé
<code>total_products</code>	float	Oui	Total produits HT

Champ	Type	Requis	Description
<code>total_products_wt</code>	float	Oui	Total produits TTC
<code>total_shipping</code>	float	Non	Frais de port
<code>total_discounts</code>	float	Non	Total des réductions
<code>conversion_rate</code>	float	Oui	Taux de conversion devise
<code>reference</code>	string	Auto	Référence commande (ex: "XKBKNABJK")
<code>invoice_number</code>	int	Non	Numéro de facture
<code>delivery_number</code>	int	Non	Numéro de bon de livraison
<code>valid</code>	0/1	Non	Commande validée ?
<code>shipping_number</code>	string	Non	Numéro de suivi colis
<code>note</code>	string	Non	Note interne
<code>date_add</code>	datetime	Auto	Date de création

Association `order_rows` (les lignes de la commande) :

Chaque commande contient une ou plusieurs lignes. Chaque ligne = un produit commandé :

Champ de la ligne	Description
<code>product_id</code>	ID du produit commandé
<code>product_attribute_id</code>	ID de la déclinaison (0 si pas de déclinaison)
<code>product_quantity</code>	Quantité commandée
<code>product_name</code>	Nom du produit (lecture seule)
<code>product_reference</code>	Référence (lecture seule)
<code>product_price</code>	Prix unitaire (lecture seule)
<code>unit_price_tax_incl</code>	Prix unitaire TTC (lecture seule)
<code>unit_price_tax_excl</code>	Prix unitaire HT (lecture seule)

Important : Les lignes de commande sont en **lecture seule** (setter=false). Tu ne peux pas modifier directement les lignes d'une commande existante via l'API orders.

Toutes les entités liées aux commandes :

Fichier	Classe	Ressource API	Rôle
<code>classes/order/Order.php</code>	<code>Order</code>	<code>orders</code>	La commande elle-même
<code>classes/order/OrderDetail.php</code>	<code>OrderDetail</code>	<code>order_details</code>	Chaque ligne de la commande
<code>classes/order/OrderHistory.php</code>	<code>OrderHistory</code>	<code>order_histories</code>	Historique des changements de statut
<code>classes/order/OrderCarrier.php</code>	<code>OrderCarrier</code>	<code>order_carriers</code>	Transporteur de la commande
<code>classes/order/OrderCartRule.php</code>	<code>OrderCartRule</code>	<code>order_cart_rules</code>	Promos/réductions appliquées
<code>classes/order/OrderInvoice.php</code>	<code>OrderInvoice</code>	<code>order_invoices</code>	Factures générées
<code>classes/order/OrderPayment.php</code>	<code>OrderPayment</code>	<code>order_payments</code>	Paiements effectués
<code>classes/order/OrderSlip.php</code>	<code>OrderSlip</code>	<code>order_slip</code>	Avoirs (remboursements)

Fichier	Classe	Ressource API	Rôle
<code>classes/order/OrderState.php</code>	<code>OrderState</code>	<code>order_states</code>	Liste des statuts possibles

Les statuts de commande (order_states)

Les statuts de commande par défaut dans PrestaShop :

ID	Nom	Description
1	En attente du paiement par chèque	Le client a choisi chèque, on attend
2	Paiement accepté	L'argent est reçu
3	Préparation en cours	On prépare le colis
4	Expédié	Le colis est parti
5	Livré	Le client a reçu son colis
6	Annulé	La commande est annulée
7	Remboursé	L'argent a été rendu
8	Erreur de paiement	Le paiement a échoué
9	En attente de virement bancaire	On attend le virement
10	En attente de paiement PayPal	On attend PayPal

Pour changer le statut d'une commande via l'API, tu modifies le champ `current_state` :

```
PATCH /api/orders/1
<prestashop><order><id>1</id><current_state>4</current_state></order></prestashop>
```

Ça appelle automatiquement `setWsCurrentState()` qui gère la logique métier (envoi d'email, génération de facture, etc.).

Fichiers liés :

<code>classes/order/Order.php</code>	← Modèle principal
<code>classes/order/OrderDetail.php</code>	← Lignes de commande
<code>classes/order/OrderHistory.php</code>	← Historique statuts
<code>classes/order/OrderState.php</code>	← Statuts possibles
<code>src/Core/Domain/Order/</code>	← DDD (Commands, Queries)
<code>src/Adapter/Order/</code>	← Adaptateurs
<code>src/Core/Order/</code>	← Services commande

11. PANIERS — Tout savoir sur l'API Carts

Vue d'ensemble

Info	Valeur
URL	<code>/api/carts</code>
Classe	<code>CartCore</code> → fichier <code>classes/Cart.php</code> (5381 lignes)
Table BDD	<code>ps_cart</code> + <code>ps_cart_product</code>
Clé primaire	<code>id_cart</code>

Un panier c'est quoi ?

Le panier c'est ce que le client remplit AVANT de payer. Il contient :

- Le client associé
- Les produits et quantités
- L'adresse de livraison choisie
- La devise et la langue

Quand le client **valide et paye**, le panier se transforme en **commande** (Order).

Tous les champs

Champ	Type	Description
<code>id</code>	int	Identifiant unique du panier
<code>id_address_delivery</code>	int	Adresse de livraison → <code>/api/addresses</code>
<code>id_address_invoice</code>	int	Adresse de facturation → <code>/api/addresses</code>
<code>id_currency</code>	int	Devise → <code>/api/currencies</code>
<code>id_customer</code>	int	Client → <code>/api/customers</code>
<code>id_guest</code>	int	Visiteur (si pas connecté) → <code>/api/guests</code>
<code>id_lang</code>	int	Langue → <code>/api/languages</code>

Association `cart_rows` (les lignes du panier) :

Chaque panier contient une ou plusieurs lignes. Chaque ligne = un produit ajouté :

Champ	Requis	Description
<code>id_product</code>	Oui	ID du produit → <code>/api/products</code>
<code>id_product_attribute</code>	Oui	ID de la déclinaison (0 si aucune) → <code>/api/combinations</code>
<code>id_address_delivery</code>	Oui	Adresse de livraison pour ce produit
<code>id_customization</code>	Non	Personnalisation éventuelle
<code>quantity</code>	Oui	Nombre d'unités

Comment fonctionne le flux Panier → Commande :

1. Créer un panier vide → POST `/api/carts`
2. Ajouter des produits → PUT `/api/carts/{id}` (avec `cart_rows`)
3. Le client paye → (géré côté front, pas via l'API directement)
4. La commande est créée → Le champ `id_cart` de la commande pointe vers ce panier

12. Le moteur interne : les classes Webservice

Dossier : `prestashop/classes/webservice/`

C'est ici que toute la magie de l'API se passe. Voici chaque fichier avec son rôle détaillé :

Fichier	Rôle détaillé
<code>WebserviceRequest.php</code>	LE CERVEAU de l'API. Gère TOUT le cycle de vie d'une requête : authentification, identification de la ressource, exécution CRUD, construction de la réponse. C'est la classe la plus importante (1947 lignes).
<code>WebserviceKey.php</code>	Modèle ObjectModel pour les clés API. Table <code>ps_webservice_account</code> . Gère la création, validation et vérification des clés.
<code>WebserviceOutputBuilder.php</code>	Construit la structure de la réponse : headers HTTP, organisation des données (vue liste ou vue détail), gestion de la profondeur.

Fichier	Rôle détaillé
<code>WebserviceOutputXML.php</code>	Séréalise les données en XML avec les namespaces xlink.
<code>WebserviceOutputJSON.php</code>	Séréalise les données en JSON.
<code>WebserviceOutputInterface.php</code>	Interface PHP que <code>WebserviceOutputXML</code> et <code>WebserviceOutputJSON</code> doivent implémenter. Garantit que les deux formats fonctionnent de la même façon.
<code>WebserviceException.php</code>	Classe d'exception spécifique au webservice, avec types DID_YOU_MEAN et SIMPLE.
<code>SQLUtils.php</code>	Utilitaires pour construire les clauses WHERE des requêtes SQL (filtres).
<code>WebserviceSpecificManagementImages.php</code>	Gestion spéciale pour l'upload/download d'images (pas un ObjectModel classique).
<code>WebserviceSpecificManagementAttachments.php</code>	Gestion spéciale pour les pièces jointes.
<code>WebserviceSpecificManagementSearch.php</code>	Gestion spéciale pour la recherche dans le catalogue.
<code>WebserviceSpecificManagementInterface.php</code>	Interface pour les gestions spéciales ci-dessus.

Le flux détaillé dans `WebserviceRequest::fetch()` :

```

fetch($key, $method, $url, $params, $bad_class_name, $inputXml)
|
| 1. isActivated()
|   → Vérifie dans ps_configuration que PS_WEBSERVICE est activé
|   → Si désactivé → erreur 503
|
| 2. authenticate()
|   → Cherche la clé dans ps_webservice_account
|   → Vérifie qu'elle existe ET qu'elle est active
|   → Si non → erreur 401
|
| 3. groupShopExists() + shopExists()
|   → Vérifie le contexte multi-boutique
|   → (pas de souci en mono-boutique)
|
| 4. shopHasRight()
|   → Charge les permissions depuis ps_webservice_permission
|   → Vérifie que cette clé a le droit sur cette ressource + méthode
|
| 5. checkResource()
|   → Parse l'URL : /api/products/1 → resource="products", id=1
|   → Vérifie que "products" existe dans getResources()
|
| 6. checkHTTPMethod()
|   → Vérifie que la méthode (GET/POST...) est autorisée pour cette ressource
|   → Ex: search interdit DELETE
|
| 7. Exécution selon la méthode :
|   | GET/HEAD → executeEntityGetAndHead() → SELECT en BDD
|   | POST     → executeEntityPost()      → INSERT en BDD via saveEntityFromXml()
|   | PUT      → executeEntityPut()        → UPDATE (tous les champs) via saveEntityFromXml()
|   | PATCH    → executeEntityPatch()      → UPDATE (champs fournis) via saveEntityFromXml()
|   | DELETE   → executeEntityDelete()     → DELETE en BDD
|
| 8. returnOutput()
|   → Construit les headers (Content-Type, statut, temps d'exécution)
|   → Appelle WebserviceOutputBuilder pour formater le contenu
|   → Retourne ['headers' => [...], 'content' => '<xml>...', 'type' => 'xml']

```

La méthode `saveEntityFromXml()` (pour POST/PUT/PATCH) :

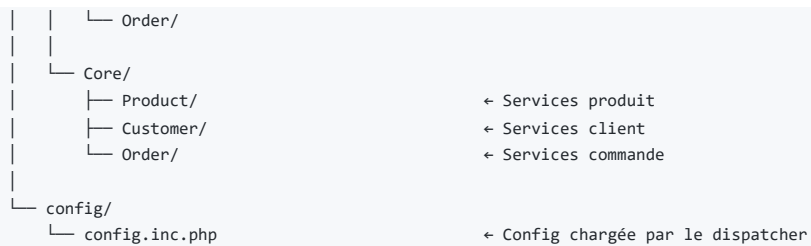
C'est la méthode qui crée ou modifie une entité à partir du XML envoyé :

1. Parse le XML avec `SimpleXMLElement`
2. Pour chaque entité dans le XML :
 - o POST → crée un nouvel objet vide

- PUT/PATCH → charge l'objet existant depuis la BDD
3. Pour chaque champ du XML :
- Vérifie s'il existe dans `$webserviceParameters`
 - Applique le setter si défini (ex: `setWsPasswd` pour le mot de passe)
 - Sinon affecte directement la propriété
4. Valide l'objet (`validateFields()`)
5. Sauvegarde en BDD (`add()` ou `update()`)
6. Traite les associations (catégories, images...)
-

13. Arborescence complète des fichiers

```
prestashop/
├── webservice/
│   └── dispatcher.php                ← POINT D'ENTRÉE de toute requête API
├── classes/
│   ├── webservice/
│   │   ├── WebserviceRequest.php    ← MOTEUR DE L'API
│   │   ├── WebserviceKey.php        ← Cerveau : auth, routing, CRUD, réponse
│   │   ├── WebserviceOutputBuilder.php ← Modèle clé API (table ps_webservice_account)
│   │   ├── WebserviceOutputXML.php  ← Construction de la réponse
│   │   ├── WebserviceOutputJSON.php ← Rendu XML
│   │   ├── WebserviceOutputInterface.php ← Rendu JSON
│   │   ├── WebserviceException.php  ← Interface commune XML/JSON
│   │   ├── SQLUtils.php             ← Exceptions
│   │   ├── WebserviceSpecificManagementImages.php ← Utilitaires SQL (filtres)
│   │   ├── WebserviceSpecificManagementAttachments.php ← Gestion images
│   │   ├── WebserviceSpecificManagementSearch.php ← Gestion pièces jointes
│   │   └── WebserviceSpecificManagementInterface.php ← Gestion recherche
│   ├── Product.php                  ← Interface gestions spéciales
│   ├── Customer.php                  ← MODÈLE PRODUIT (8491 lignes)
│   ├── Cart.php                      ← MODÈLE CLIENT (1566 lignes)
│   ├── Category.php                  ← MODÈLE PANIER (5381 lignes)
│   ├── Carrier.php                   ← Modèle Catégorie
│   ├── Address.php                   ← Modèle Transporteur
│   ├── Manufacturer.php               ← Modèle Adresse
│   ├── Supplier.php                  ← Modèle Marque
│   └── Combination.php                ← Modèle Fournisseur
│
│   ├── order/
│   │   ├── Order.php                 ← MODÈLE DÉCLINAISON
│   │   ├── OrderDetail.php           ← TOUT CE QUI TOUCHE AUX COMMANDES
│   │   ├── OrderHistory.php          ← Modèle Commande (2877 lignes)
│   │   ├── OrderCarrier.php          ← Lignes de commande
│   │   ├── OrderCartRule.php         ← Historique des statuts
│   │   ├── OrderInvoice.php          ← Transporteur de la commande
│   │   ├── OrderPayment.php          ← Promos appliquées
│   │   ├── OrderSlip.php              ← Factures
│   │   └── OrderState.php             ← Paiements
│   │
│   ├── stock/
│   │   └── StockAvailable.php         ← Avoirs/Remboursements
│   │
│   └── tax/
│       ├── Tax.php                   ← Statuts possibles
│       ├── TaxRule.php
│       └── TaxRulesGroup.php
│
├── controllers/admin/                ← CONTROLLERS BACK OFFICE
│   ├── AdminProductsController.php
│   ├── AdminCartsController.php
│   ├── AdminCartRulesController.php
│   ├── AdminCustomerThreadsController.php
│   └── BoOrder.php
│
├── src/
│   ├── Core/Domain/
│   │   ├── Product/                  ← NOUVEAU SYSTÈME (Symfony / DDD)
│   │   ├── Customer/
│   │   ├── Order/
│   │   ├── OrderState/
│   │   ├── OrderReturn/
│   │   └── CustomerService/
│   │
│   └── Adapter/
│       ├── Product/
│       └── Customer/
│
└──
```



14. Exemples concrets de requêtes (copier-coller)

Lister tous les produits :

```
GET http://localhost:8080/api/products?ws_key=VOTRE_CLE
```

Obtenir UN produit (id=1) avec TOUS ses champs :

```
GET http://localhost:8080/api/products/1?ws_key=VOTRE_CLE&display=full
```

Obtenir seulement l'id, le nom et le prix des produits :

```
GET http://localhost:8080/api/products?ws_key=VOTRE_CLE&display=[id,name,price]
```

Obtenir le schéma XML vierge d'un produit (pour savoir quels champs envoyer) :

```
GET http://localhost:8080/api/products?schema=blank&ws_key=VOTRE_CLE
```

Filtrer les produits actifs :

```
GET http://localhost:8080/api/products?ws_key=VOTRE_CLE&filter[active]=1
```

Filtrer par range de prix (entre 10 et 50 euros) :

```
GET http://localhost:8080/api/products?ws_key=VOTRE_CLE&filter[price]=[10,50]
```

Trier par nom croissant :

```
GET http://localhost:8080/api/products?ws_key=VOTRE_CLE&sort=[name_ASC]
```

Pagination (10 résultats à partir du 5ème) :

```
GET http://localhost:8080/api/products?ws_key=VOTRE_CLE&limit=5,10
```

Recevoir en JSON au lieu de XML :

```
GET http://localhost:8080/api/products?ws_key=VOTRE_CLE&output_format=JSON
```

Créer un nouveau client :

```
POST http://localhost:8080/api/customers?ws_key=VOTRE_CLE
Content-Type: application/xml

<prestashop>
  <customer>
    <firstname>Jean</firstname>
    <lastname>Dupont</lastname>
    <email>jean.dupont@example.com</email>
    <passwd>MonMotDePasse123</passwd>
    <id_default_group>3</id_default_group>
    <associations>
      <groups>
        <group><id>3</id></group>
      </groups>
    </associations>
  </customer>
</prestashop>
```

Modifier le prix d'un produit (PATCH = modification partielle) :

```
PATCH http://localhost:8080/api/products/1?ws_key=VOTRE_CLE
Content-Type: application/xml

<prestashop>
  <product>
    <id>1</id>
    <price>19.99</price>
  </product>
</prestashop>
```

Changer le statut d'une commande (passer à "Expédié") :

```
PATCH http://localhost:8080/api/orders/1?ws_key=VOTRE_CLE
Content-Type: application/xml

<prestashop>
  <order>
    <id>1</id>
    <current_state>4</current_state>
  </order>
</prestashop>
```

Supprimer un produit :

```
DELETE http://localhost:8080/api/products/5?ws_key=VOTRE_CLE
```

Tous les paramètres de filtrage/affichage :

Paramètre	Exemple	Ce que ça fait
display	display=full	Affiche TOUS les champs
display	display=[id,name,price]	Affiche seulement ces champs
filter[champ]	filter[active]=1	Filtre par valeur exacte
filter[champ]	filter[price]=[10,50]	Filtre par range
filter[champ]	filter[name]=%shirt%	Filtre par texte (LIKE)
sort	sort=[name_ASC]	Tri croissant par nom
sort	sort=[price_DESC]	Tri décroissant par prix

Paramètre	Exemple	Ce que ça fait
<code>limit</code>	<code>limit=10</code>	Limiter à 10 résultats
<code>limit</code>	<code>limit=5,10</code>	10 résultats à partir du 5ème
<code>language</code>	<code>language=1</code>	Filtrer par langue (id=1 = français)
<code>schema</code>	<code>schema=blank</code>	Obtenir un template XML vide
<code>schema</code>	<code>schema=synopsis</code>	Obtenir la description des champs
<code>output_format</code>	<code>output_format=JSON</code>	Recevoir du JSON
<code>date</code>	<code>date=1</code>	Inclure les champs de date

15. Scénarios complets pour la NewApp

Voici les scénarios typiques que ta NewApp devra implémenter :

Scénario 1 : Afficher le catalogue de produits

1. GET `/api/categories?display=full&output_format=JSON`
→ Récupère toutes les catégories pour le menu
2. GET `/api/products?display=[id,name,price,id_default_image,description_short]&filter[active]=1&output_format=JSON`
→ Récupère tous les produits actifs avec les infos de base
3. GET `/api/images/products/{id_product}/{id_image}`
→ Récupère l'image de chaque produit

Scénario 2 : Afficher la fiche d'un produit

1. GET `/api/products/{id}?display=full&output_format=JSON`
→ Récupère TOUS les détails du produit
2. GET `/api/product_option_values?filter[id]=[1|2|3]&output_format=JSON`
→ Récupère les déclinaisons (tailles, couleurs)
3. GET `/api/stock_availability?filter[id_product]={id}&output_format=JSON`
→ Récupère le stock par déclinaison

Scénario 3 : Créer un compte client

1. POST `/api/customers`
→ Envoie `firstname, lastname, email, passwd`
→ Reçoit le client créé avec son `id`
2. POST `/api/addresses`
→ Envoie l'adresse du client (`id_customer, address1, city, postcode, id_country`)

Scénario 4 : Ajouter au panier et commander

1. POST `/api/carts`
→ Crée un panier vide avec `id_customer, id_currency, id_lang`
2. PUT `/api/carts/{id}`
→ Ajoute des produits (`cart_rows` avec `id_product, quantity`)
3. POST `/api/orders`
→ Crée la commande à partir du panier
→ Envoie : `id_cart, id_customer, id_address_delivery, id_carrier, module, payment, total_paid...`

Scénario 5 : Suivre les commandes d'un client

1. GET /api/orders?filter[id_customer]={id}&display=full&output_format=JSON
→ Liste toutes les commandes du client
2. GET /api/order_histories?filter[id_order]={id}&sort=[date_add_DESC]&output_format=JSON
→ Historique des statuts de la commande
3. GET /api/order_details?filter[id_order]={id}&output_format=JSON
→ Détails des produits commandés